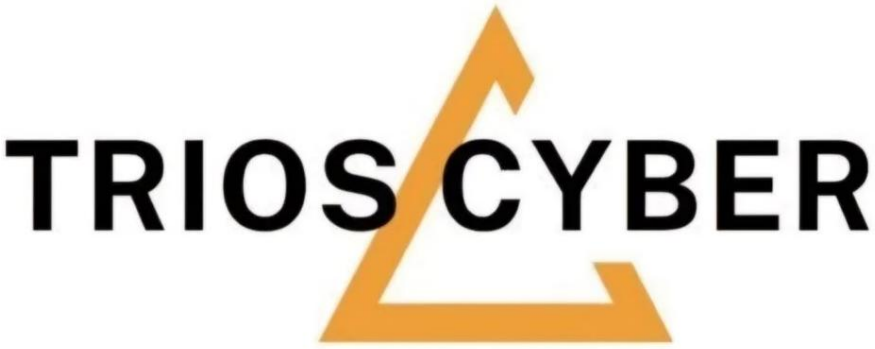


TRIOS CYBER

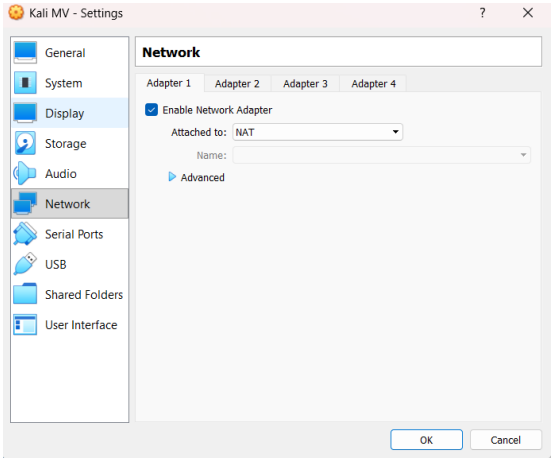
LABORATORY PRACTICE 3

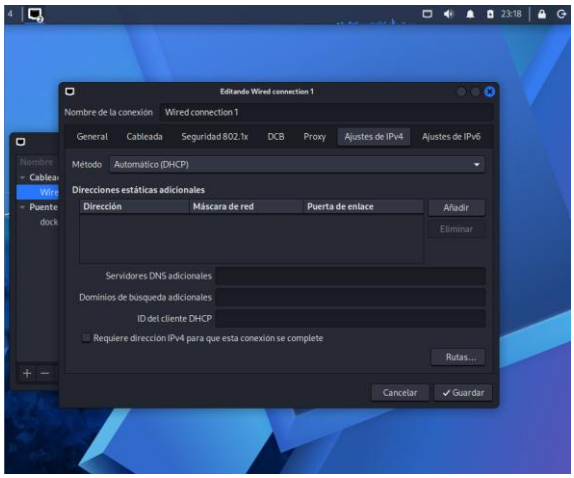
Wireshark Traffic Capture and Protocol Analysis

Name: Matthew Josue Portilla Morejon

1. Kali Linux NAT Network Setup

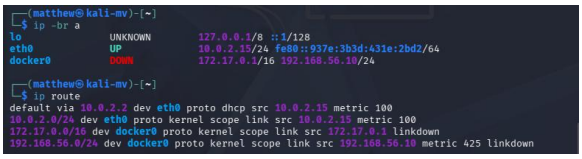
The Kali Linux virtual machine was configured in Oracle VirtualBox using NAT networking. IPv4 addressing was set to automatic DHCP so the virtual machine could obtain the network parameters required for the laboratory.

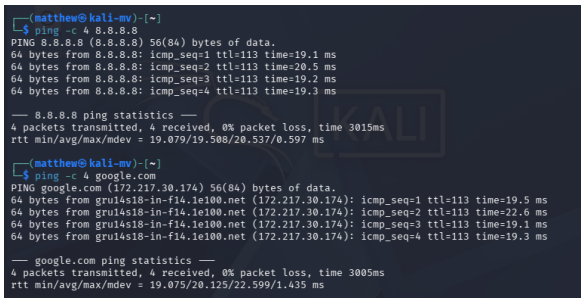




2. Network and Internet Verification

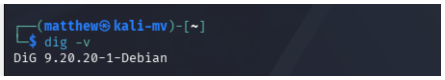
The network configuration was restarted and verified from Kali Linux. The active interface eth0 received the IPv4 address 10.0.2.15/24 and a default route through 10.0.2.2. Internet reachability and DNS resolution were then verified using connectivity tests.





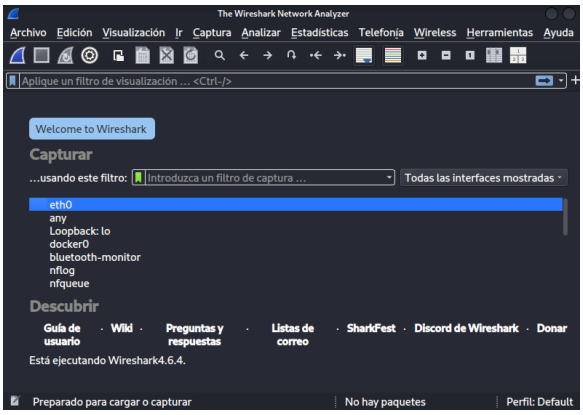
3. Wireshark and dig Verification

Wireshark and dig were verified on the Kali Linux system before the packet analysis activity. These tools were used to inspect captured packets and generate DNS lookup traffic.



4. Traffic Capture Preparation

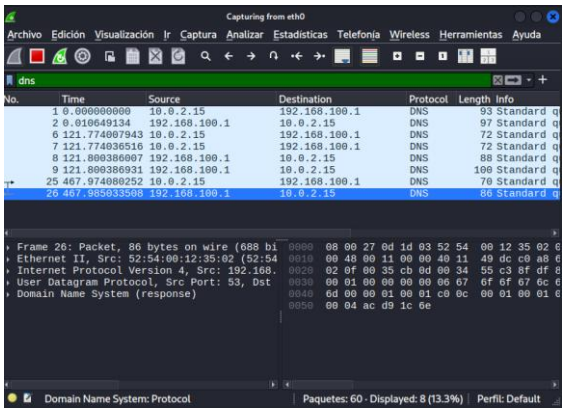
Wireshark was prepared to analyze traffic from the active network interface. Traffic from eth0 was also written to a PCAP file so that the generated laboratory packets could be reviewed with Wireshark.

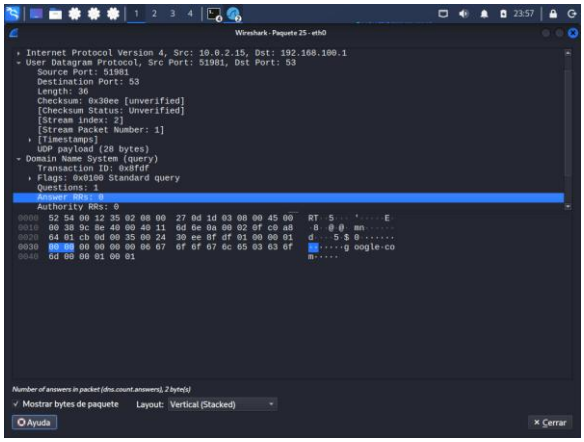


5. DNS Lookup and DNS Traffic Analysis

A DNS lookup for google.com was generated using dig and the resulting DNS packets were filtered in Wireshark. The request originated from 10.0.2.15 and was sent to 192.168.100.1 using UDP source port 51981 and destination port 53. The corresponding DNS response was also observed.

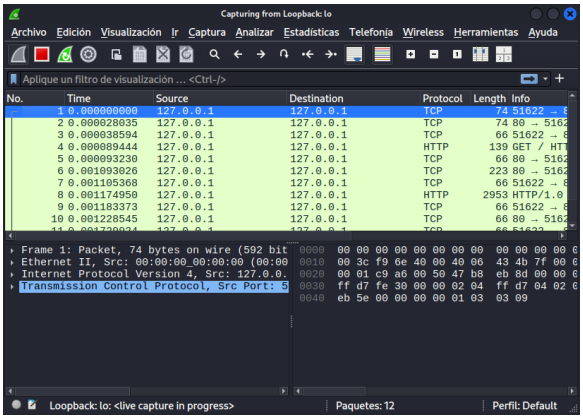
```
┌──(matthew㉿kali-mv)-[~]
└─$ dig google.com
; <<>> DiG 9.20.20-1-Debian <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 15786
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             226     IN      A       172.217.30.174

;; Query time: 8 msec
;; SERVER: 192.168.100.1#53(192.168.100.1) (UDP)
;; WHEN: Fri Sep 04 23:40:28 -05 2026
;; MSG SIZE  rcvd: 55
```

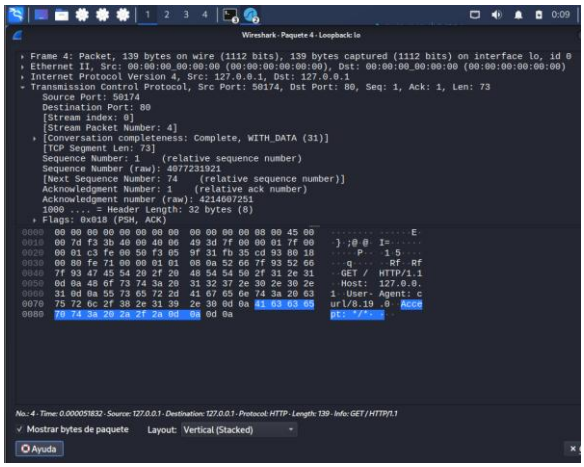




6. HTTP Traffic Generation and Analysis

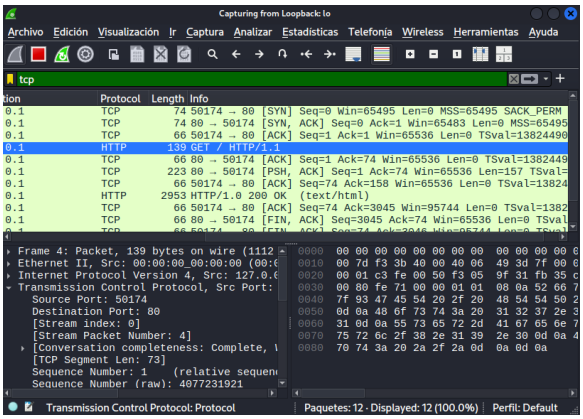
A local HTTP service was used on the Kali Linux loopback interface to generate clear-text web traffic. The HTTP request was captured between 127.0.0.1 and 127.0.0.1 using TCP source port 50174 and destination port 80. Wireshark displayed the GET / HTTP/1.1 request directly because the HTTP communication was not encrypted.

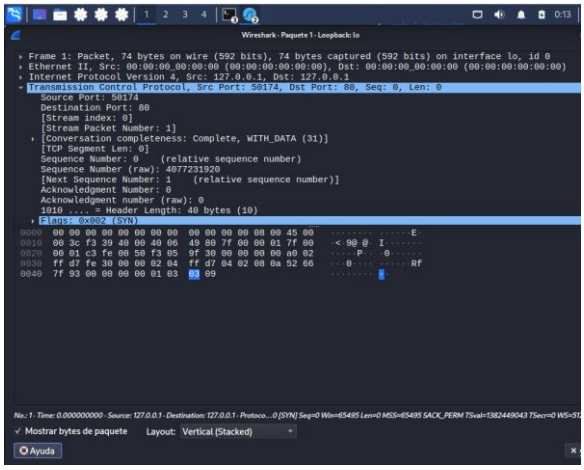




7. TCP Connection Analysis

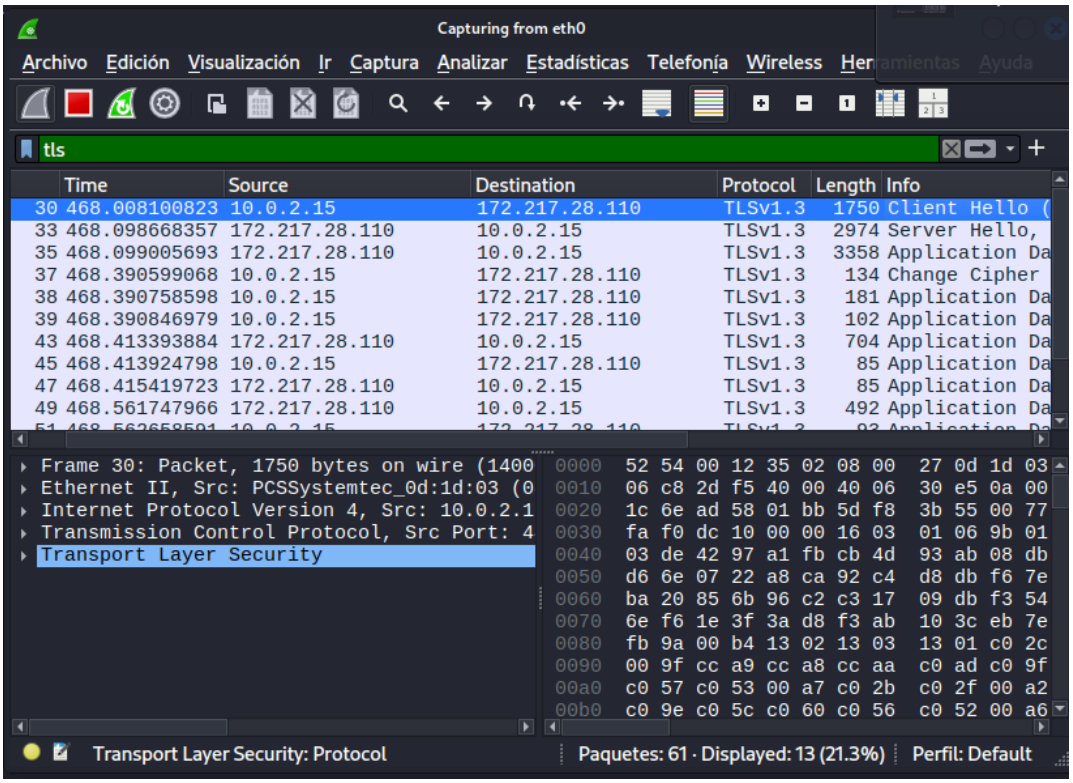
The TCP session used by the HTTP request was reviewed in Wireshark. The connection establishment showed the expected three-way handshake: SYN from source port 50174 to destination port 80, SYN-ACK from port 80 back to 50174, and the final ACK before the HTTP GET request was transmitted.

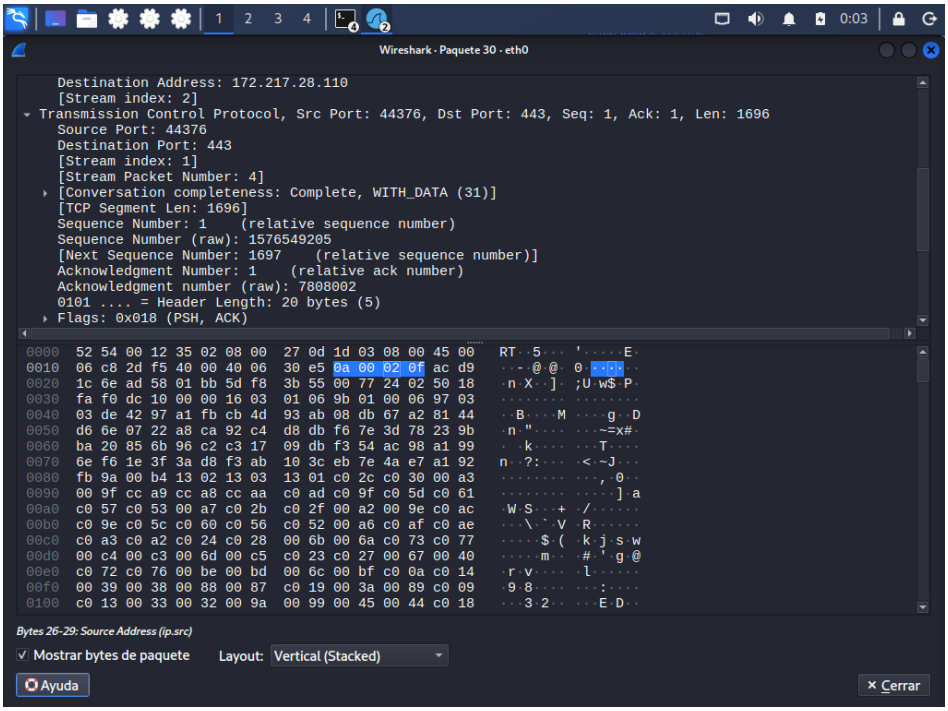




8. HTTPS/TLS Traffic Generation and Analysis

HTTPS traffic was generated from Kali Linux and filtered as TLS traffic in Wireshark. The observed connection originated from 10.0.2.15 and was sent to 172.217.28.110 using TCP source port 44376 and destination port 443. TLS 1.3 Client Hello and encrypted application-data packets were observed.





9. Protocol Behavior Review

The captured traffic showed the expected behavior of the analyzed protocols. DNS used UDP port 53 to resolve a domain name; TCP established a session with SYN, SYN-ACK, and ACK messages; HTTP used TCP port 80 and

exposed the GET request in readable form; and HTTPS used TCP port 443 with TLS 1.3, leaving the application content encrypted while source, destination, ports, and TLS handshake information remained visible.

10. Conclusions

The Wireshark activity was successfully completed using traffic generated from the Kali Linux laboratory. DNS, TCP, HTTP, and HTTPS/TLS traffic were identified and filtered, and the corresponding source and destination addresses, ports, and protocol behavior were reviewed. The results demonstrated the difference between unencrypted HTTP traffic and TLS-protected HTTPS communication while confirming how Wireshark can be used to analyze network behavior within the authorized laboratory environment.